LeetCode 1356: Sort Integers by The Number of 1 Bits

1. Problem Title & Link

Title: LeetCode 1356: Sort Integers by The Number of 1 Bits

Link: https://leetcode.com/problems/sort-integers-by-the-number-of-1-bits/

2. Problem Statement (Short Summary)

Given an integer array arr, sort the integers based on:

1. The number of 1s in their binary representation (**ascending**).
2. If two numbers have the same number of 1s, sort them by their value (**ascending**).

Return the sorted array.

Example

Input:

arr = [0,1,2,3,4,5,6,7,8]

Binary representations:

0 -> 0000 -> 0 ones

1 -> 0001 -> 1 one

2 -> 0010 -> 1 one

3 -> 0011 -> 2 ones

4 -> 0100 -> 1 one

5 -> 0101 -> 2 ones

6 -> 0110 -> 2 ones

7 -> 0111 -> 3 ones

8 -> 1000 -> 1 one

Output:

[0,1,2,4,8,3,5,6,7]

3. Examples (Input → Output)

Example 1

Input:

arr = [0,1,2,3,4,5,6,7,8]

Output:

[0,1,2,4,8,3,5,6,7]

Example 2

Input:

arr = [1024,512,256,128,64,32,16,8,4,2,1]

Output:

[1,2,4,8,16,32,64,128,256,512,1024]

All have exactly one set bit.

Example 3

Input:

arr = [10,100,1000,10000]

Output:

[10,100,10000,1000]

4. Constraints

1 <= arr.length <= 500

0 <= arr[i] <= 10^4

Important:

Sort by:

1. Bit count
2. Number value

5. Core Concept (Pattern / Topic)

Bit Manipulation + Sorting

Secondary Topics:

- Custom Sorting
- Binary Representation

6. Thought Process (Step-by-Step Explanation)

Understanding the Problem

We need to count:

Number of 1 bits

in each number's binary form.

Example:

5

Binary:

101

Contains:

2 ones

Example

Array:

[5,3]

Binary:

5 = 101 -> 2 ones

3 = 11 -> 2 ones

Both have same bit count.

So compare values:

3 < 5

Result:

[3,5]

Brute Force Idea

For each number:

1. Convert to binary.
2. Count ones.
3. Sort using custom rule.

Works.

Key Observation

Every number gets a ranking:

(bitCount, value)

Example:

5 -> (2,5)

3 -> (2,3)

8 -> (1,8)

Sort these pairs.

Why This Works

Sorting automatically compares:

First: bit count

Then: value

Exactly matching problem requirements.

7. Visual / Intuition Diagram (ASCII Diagram)

```
Input:
[5,3,8]
Convert:
5 -> (2,5)

3 -> (2,3)

8 -> (1,8)
Sort:
(1,8)

(2,3)

(2,5)
Result:
[8,3,5]
```

8. Pseudocode (Language Independent)

```
function bitCount(num)

    count = 0

    while num > 0

        count += num % 2

        num = num / 2

    return count

sort arr using:

(bitCount(number), number)

return arr
```

9. Code Implementation

Python

```
class Solution:
    def sortByBits(self, arr):

        return sorted(
            arr,
            key=lambda x: (
```

```
                bin(x).count('1'),
                x
            )
        )
```

Java

```
import java.util.*;

class Solution {

    public int[] sortByBits(int[] arr) {

        Integer[] nums =
            Arrays.stream(arr)
                  .boxed()
                  .toArray(Integer[]::new);

        Arrays.sort(nums,
            (a, b) -> {

                int bitsA =
                    Integer.bitCount(a);

                int bitsB =
                    Integer.bitCount(b);

                if(bitsA == bitsB)
                    return a - b;

                return bitsA - bitsB;
            });

        for(int i = 0;
            i < arr.length;
            i++) {

            arr[i] = nums[i];
        }

        return arr;
    }
}
```

10. Time & Space Complexity

Time Complexity

Sorting:

O(n log n)

Bit counting:

O(log M)

where:

M = maximum value

Overall:

O(n log n)

Space Complexity

Python sorting:

O(n)

Java boxed array:

O(n)

11. Common Mistakes / Edge Cases

Mistake 1

Sorting only by value.

Example:

[3,8]

Correct:

8 first

because:

8 -> 1 bit

3 -> 2 bits

Mistake 2

Sorting only by bit count.

Example:

[5,3]

Both have:

2 bits

Need value tie-breaker.

Mistake 3

Counting binary digits instead of set bits.

Example:

5 = 101

Length:

3

Bit count:

2

Different concepts.

12. Detailed Dry Run

Input:

arr = [5,3,8]

Compute:

Number	Binary	Ones	Pair
5	101	2	(2,5)
3	11	2	(2,3)
8	1000	1	(1,8)

Sort pairs:

(1,8)

(2,3)

(2,5)

Result:

[8,3,5]

13. Common Use Cases (Real-Life / Interview)

Bitmask Problems

Compare numbers by active bits.

Network Flags

Count enabled features.

Permission Systems

Compare access bitmasks.

Interview Purpose

Tests:

Bit Manipulation

Custom Sorting

Binary Representation

14. Common Traps (Important!)

Trap 1

Confusing:

Binary length

with:

Number of set bits

Trap 2

Forgetting secondary sorting rule.

Trap 3

Overcomplicating bit counting.

Python:

bin(x).count('1')

is enough.

15. Builds To (Related LeetCode Problems)

LC 191 - Number of 1 Bits

↓

LC 338 - Counting Bits

↓

LC 1356 - Sort Integers by Number of 1 Bits

↓

LC 231 - Power of Two

↓

LC 342 - Power of Four

These strengthen bit manipulation skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Convert to Binary String	O(n log n)	O(n)	Good

Brian Kernighan Bit Count	O(n log n)	O(n)	Better
Built-In Bit Count	O(n log n)	O(n)	Best

Approach 1: Binary String

bin(x).count('1')

Simple and readable.

Approach 2: Brian Kernighan Algorithm

```
while n:
   n &= (n - 1)
   count += 1
```

Removes one set bit per iteration.

Approach 3: Built-In (Chosen)

Most concise.

Most interviewers accept it.

17. Why This Solution Works (Short Intuition)

Each number is represented by:

(bitCount, value)

Sorting by these two properties exactly matches the problem's required ordering.

18. Variations / Follow-Up Questions

Follow-Up 1

Sort by number of zero bits.

Follow-Up 2

Sort descending by bit count.

Follow-Up 3

Group numbers with equal bit counts.

Follow-Up 4

Return only numbers having exactly K set bits.

Interview Takeaway

Whenever you hear:

Binary Representation

Count 1 Bits

Set Bits

Bitmask Ranking

think:

Bit Manipulation

+

Custom Sorting

A common interview trick is to compute a custom key (such as bit count) and sort using that key rather than comparing the original values directly.